

CSE 445: Machine Learning

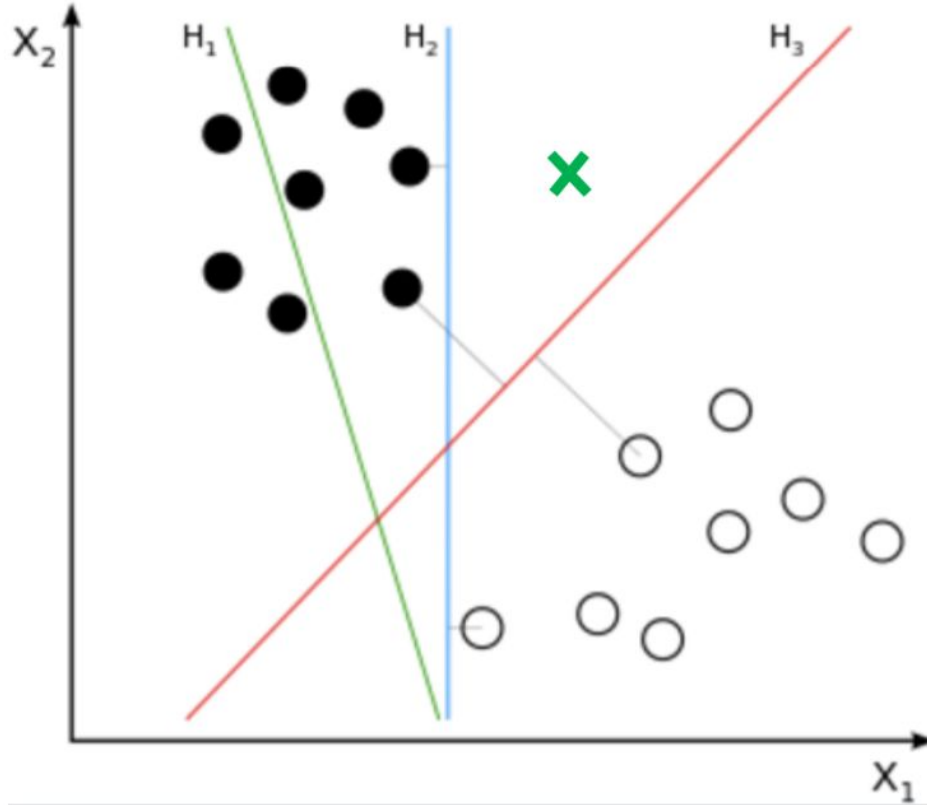
Support Vector Machines

Support Vector Machines:

Let 's have some space

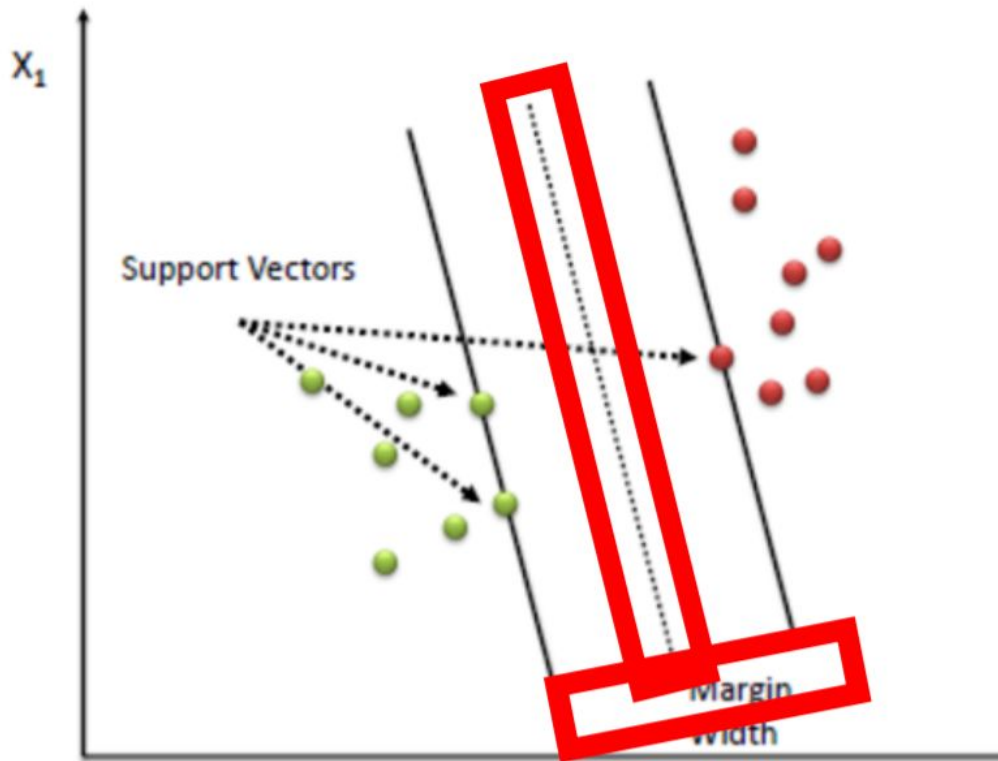


Support Vector Machine - Motivation



- To which class would each decision boundary assign the new instance?
- Which decision boundary would you choose to separate data?

Support Vector Machines - Motivation



- Choose hyperplane that maximizes the “margin” width
- Margin $\square$ distance between the decision boundary (i.e. separating hyperplane) and training instances (“support vectors”) closest to the hyperplane

Support Vector Machine

- Linear or nonlinear classification, regression, outlier detection
- Excellent for small or medium sized datasets
- Large margin classification - fit the widest possible street between the classes
- More instances “off-the-street” does not affect decision boundary

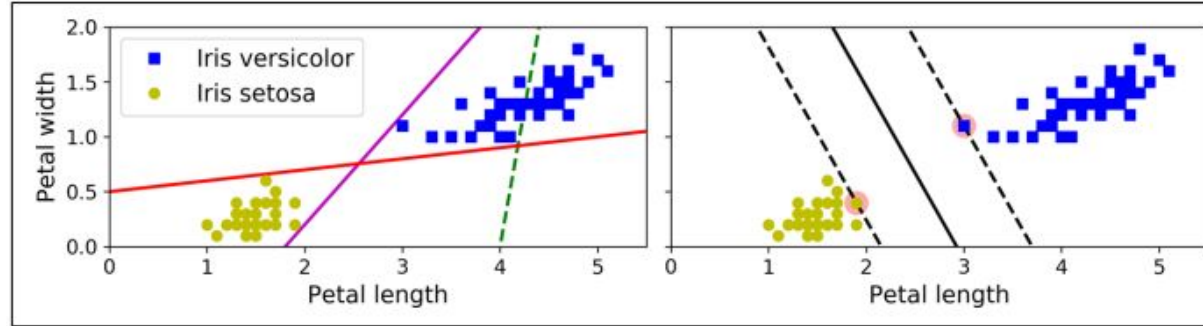


Figure 5-1. Large margin classification

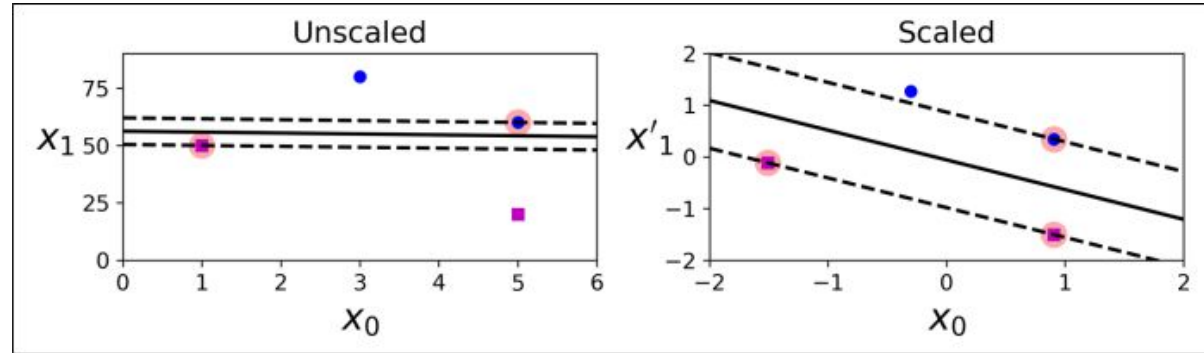


Figure 5-2. Sensitivity to feature scales

Soft Margin Classification

- Can't strictly enforce all training data to be on the correct side of the decision boundary (hard margin classification) – only works if it's linearly separable
- Adversely affected by outliers
- Keep margin as wide as possible, while limiting margin violations
- C □ hyperparameter controlling number of margin violations

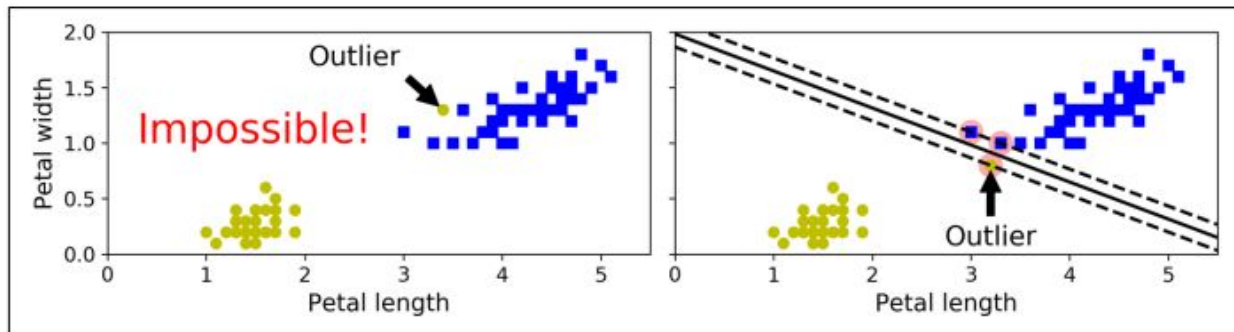


Figure 5-3. Hard margin sensitivity to outliers

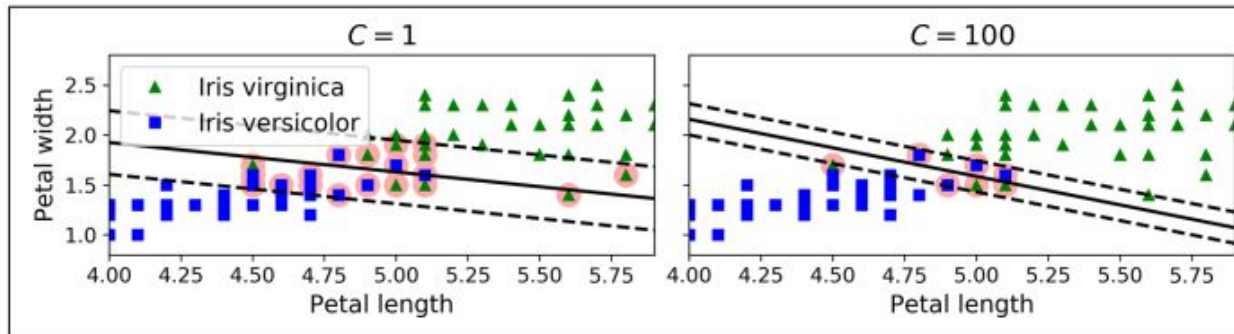


Figure 5-4. Large margin (left) versus fewer margin violations (right)

Hard Margin Classifier – Decision function

- In Linear Regression, parameter vector was denoted as θ
- For SVM, more convenient to use bias $b = \theta_0$ and feature weights $\mathbf{w} = \{\theta_1, \theta_2, \dots, \theta_n\}$
- Simply compute $\mathbf{w}^T \mathbf{x} + \mathbf{b}$ – if result is positive, predicted class is in positive class, and vice versa

$$\hat{y} = \begin{cases} 0 & \text{if } \mathbf{w}^T \mathbf{x} + \mathbf{b} < 0 \\ 1 & \text{if } \mathbf{w}^T \mathbf{x} + \mathbf{b} \geq 0 \end{cases}$$

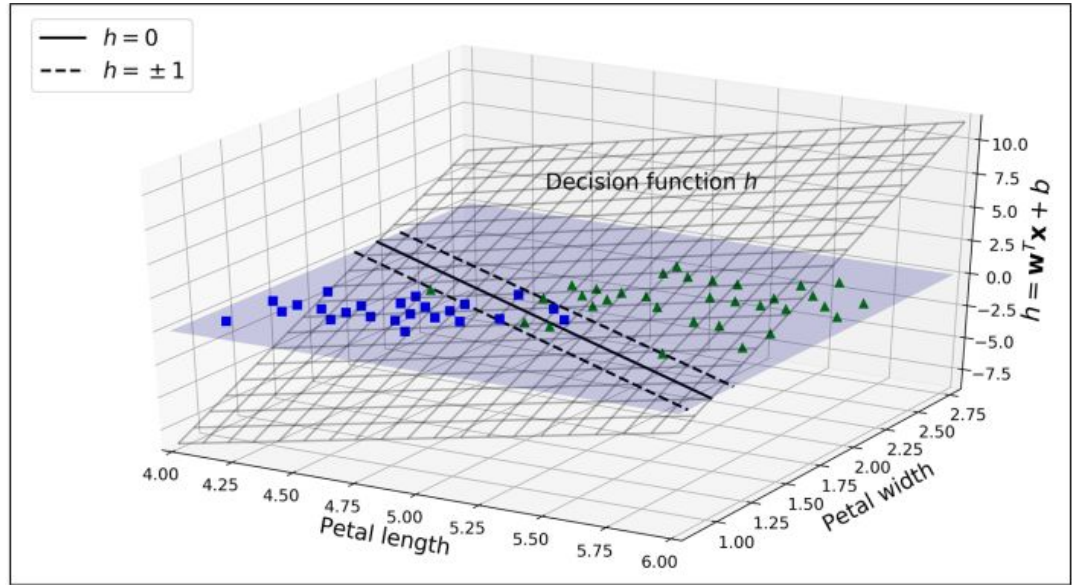


Figure 5-12. Decision function for the iris dataset

Training Objective

- Slope of the decision function is equal to the norm of the weight vector $\|\mathbf{w}\|$
- Smaller weight vector $\mathbf{w}$ means larger margin
- Minimize $\|\mathbf{w}\|$ to get a large margin, while avoiding margin violations

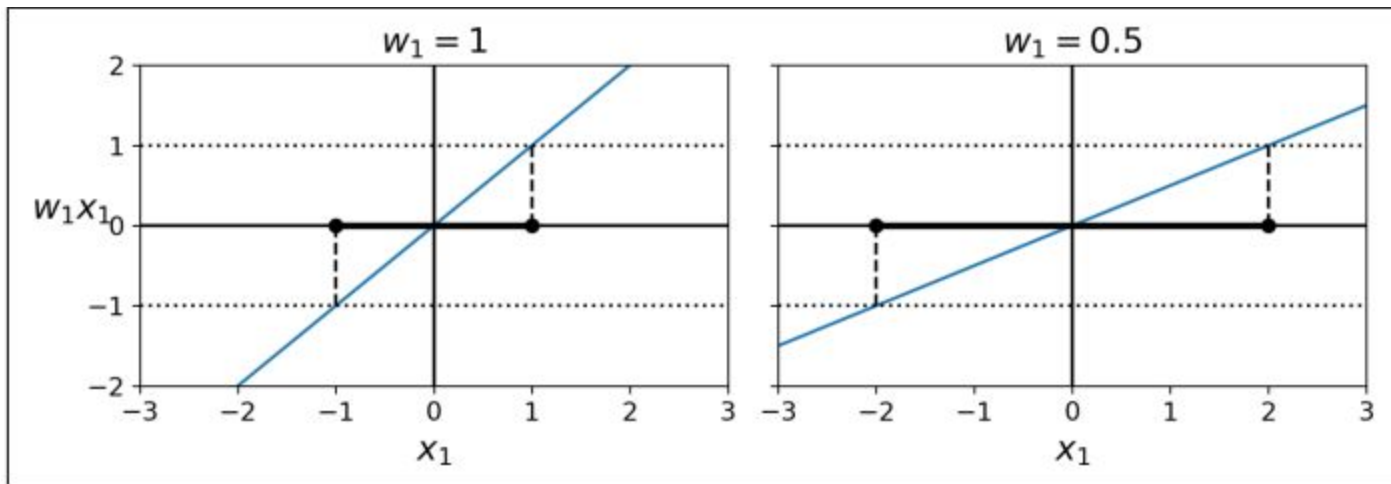


Figure 5-13. A smaller weight vector results in a larger margin

Hard Margin Classifier Objective

- Constrained Optimization problem:

$$\underset{\mathbf{w}, b}{\text{minimize}} \quad \frac{1}{2} \mathbf{w}^T \mathbf{w}$$

- Subject to the following constraint:

$$t^{(i)} (\mathbf{w}^T \mathbf{x}^{(i)} + b) \geq 1 \quad \text{for } i = 1, 2, \dots, m$$

- $t^{(i)}$ is -1 for negative instances, and +1 for positive instances
- $\mathbf{w}$ is the weight vector

Soft Margin Classifier Objective

- Introduce a slack variable $\zeta^{(i)} \geq 0$ for each instance – measure of how much the i^{th} instance is allowed to violate the margin

$$\underset{\mathbf{w}, b, \zeta}{\text{minimize}} \quad \frac{1}{2} \mathbf{w}^T \mathbf{w} + C \sum_{i=1}^m \zeta^{(i)}$$

- Subject to the following constraint:

$$t^{(i)} (\mathbf{w}^T \mathbf{x}^{(i)} + b) \geq 1 - \zeta^{(i)} \quad \text{and} \quad \zeta^{(i)} \geq 0 \quad \text{for } i = 1, 2, \dots, m$$

- C to allows us to balance the two conflicting objectives: minimize weights to increase margin, and make slack variables small to reduce margin violations.

Computational Complexity

- LinearSVC □ used for linear SVMs, and scales linearly with number of training instances and features
- SVC □ based on libsvm library, supports the kernel trick, but doesn't scale linearly with number of instances [very good for small datasets wrt number of instances]

Table 5-1. Comparison of Scikit-Learn classes for SVM classification

Class	Time complexity	Out-of-core support	Scaling required	Kernel trick
LinearSVC	$O(m \times n)$	No	Yes	No
SGDClassifier	$O(m \times n)$	Yes	Yes	No
SVC	$O(m^2 \times n)$ to $O(m^3 \times n)$	No	Yes	Yes

Support Vector Regression

- Reverse objective from classification
 - try to fit as many instances as possible on the street, while limiting margin violations
- Adding more instances within the margin does not affect predictions □ ϵ insensitive
- For nonlinear regression tasks, use kernelized SVM model in SVR

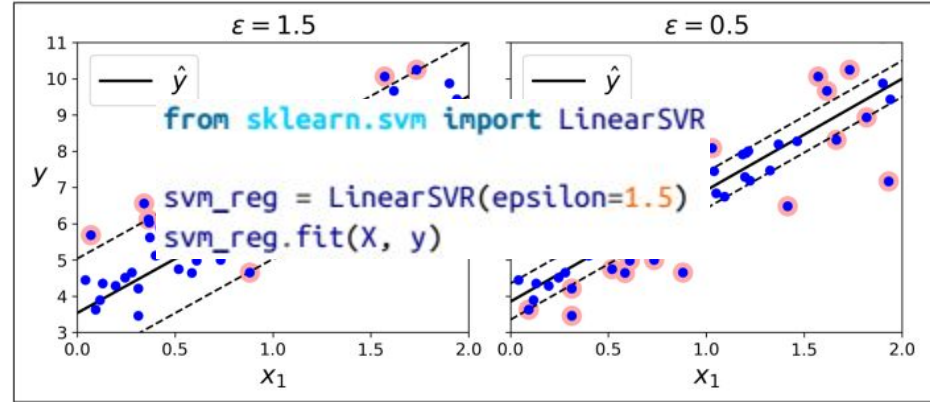


Figure 5-10. SVM Regression

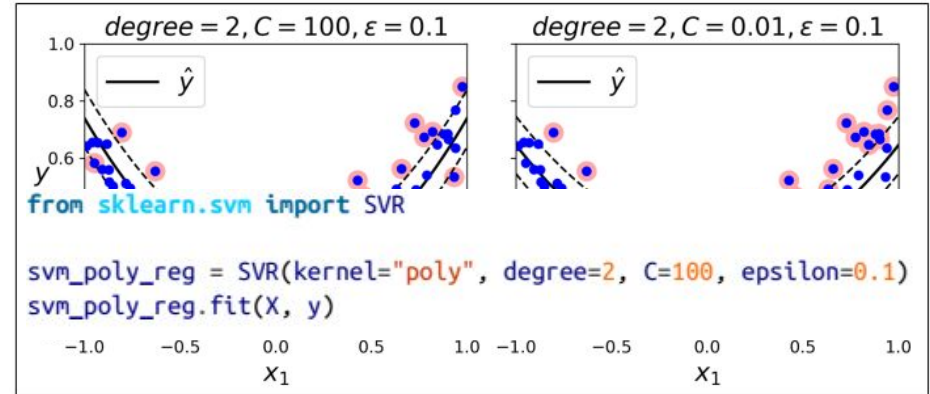
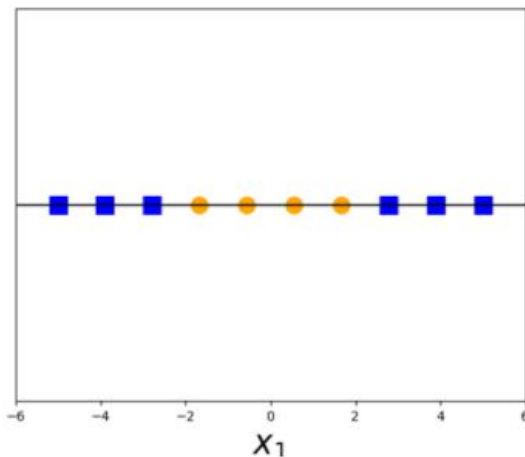


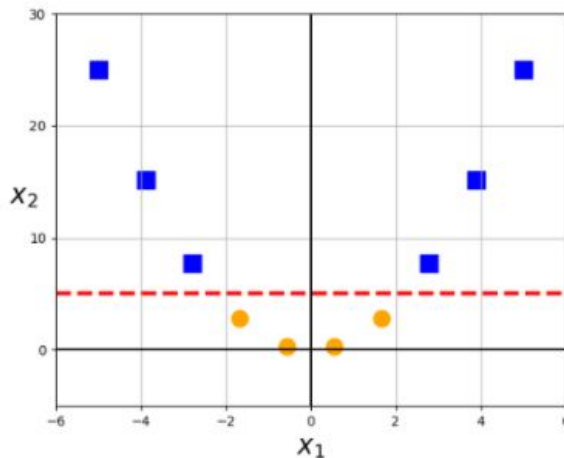
Figure 5-11. SVM Regression using a second-degree polynomial kernel

Nonlinear Transformations – 1D

- If data isn't linearly separable in the original space □ map it to a higher dimensional feature space



This data becomes linearly separable after a quadratic transformation to 2-dimensions.



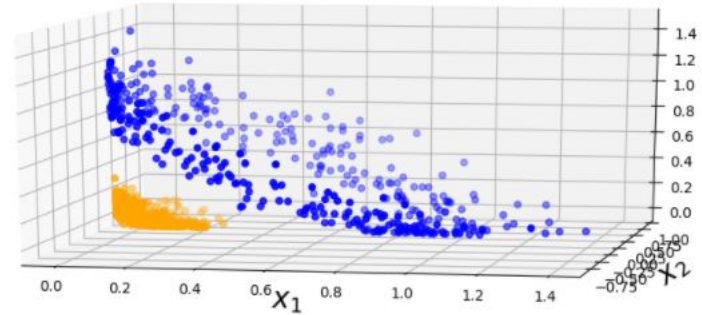
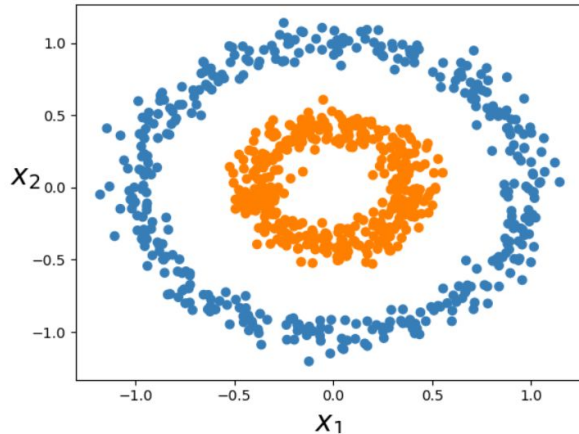
- Original data isn't linearly separable
- Apply nonlinear transformation as $\phi(x) = x^2 \rightarrow$ we can draw a linear decision boundary!
- Several functions possible to map the data to higher dimensions

Nonlinear Transformations – 2D

- Applying a second-degree polynomial mapping $\rightarrow$

$$\phi(x) = \phi\left(\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}\right) = \begin{pmatrix} x_1^2 \\ \sqrt{2}x_1x_2 \\ x_2^2 \end{pmatrix}$$

- Note: Transformed vector is 3D!



Linearly separable data in 3-d after applying the 2nd-degree polynomial transformation

Kernel Trick

- Kernel function ϕ takes vectors in the original lower dimensional space as inputs, and returns the dot product of the transformed vectors in the higher dimensional feature space
- Consider a couple of 2D vectors $\mathbf{a}, \mathbf{b}$

Equation 5-9. Kernel trick for a second-degree polynomial mapping

$$\begin{aligned}\phi(\mathbf{a})^\top \phi(\mathbf{b}) &= \begin{pmatrix} a_1^2 \\ \sqrt{2} a_1 a_2 \\ a_2^2 \end{pmatrix}^\top \begin{pmatrix} b_1^2 \\ \sqrt{2} b_1 b_2 \\ b_2^2 \end{pmatrix} = a_1^2 b_1^2 + 2a_1 b_1 a_2 b_2 + a_2^2 b_2^2 \\ &= (a_1 b_1 + a_2 b_2)^2 = \left(\begin{pmatrix} a_1 \\ a_2 \end{pmatrix}^\top \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} \right)^2 = (\mathbf{a}^\top \mathbf{b})^2\end{aligned}$$

- Dot product of transformed vectors is equal to the square of the dot product of the original vectors
- The kernel is any function that's capable of computing the dot product without having to compute (or know

Equation 5-10. Common kernels

Linear: $K(\mathbf{a}, \mathbf{b}) = \mathbf{a}^\top \mathbf{b}$

Polynomial: $K(\mathbf{a}, \mathbf{b}) = (\gamma \mathbf{a}^\top \mathbf{b} + r)^d$

Gaussian RBF: $K(\mathbf{a}, \mathbf{b}) = \exp(-\gamma \|\mathbf{a} - \mathbf{b}\|^2)$

Sigmoid: $K(\mathbf{a}, \mathbf{b}) = \tanh(\gamma \mathbf{a}^\top \mathbf{b} + r)$